



Application Note

Segger Ozone and J-Link Debug

Version 1.01

Table of Contents

Introduction	3
Prerequisites	3
Software and packages used in this guide	3
Devices used	3
Installing Ozone and J-Link	3
About Ozone	3
About J-link	3
Installing Ozone	3
Installing J-link	4
Connecting the J-Link probe	4
Configuring the J-Link probe	4
Preparing stubs for the M55_HE CPU core	5
Building the Blinky Program file	6
Debugging Blinky using Ozone and J-Link	7
Document History	12

Introduction

This application note covers the use of Segger Ozone debugger and J-Link probe to debug a project using a program file generated using Arm Development Studio or Keil MDK. The blinky project provided as part of the Alif CMSIS pack is used to demonstrate how to initiate a debug session.

Prerequisites

Before proceeding it is recommended that you are familiar with the Alif Development Kit Quick Start Guide and the Alif Security Toolkit Quick Start Guide.

Software and packages used in this guide

- Linux / Windows 10
- Arm Keil uVision5 MDK V5.36.0.0
- Arm Development Studio V2021.2
- Segger Ozone (3.26f)
- Segger J-Link (version 7.64d)
- Segger Configuration (version 7.64d)
- Alif SETOOLS (version 0.46)
- Alif Ensemble CMSIS package (version 0.2.0)

Devices used

- Alif A1 Development Kit (CPU board + base board)
- Segger J-Link Ultra + (version 5.1) & J-Link Base (version 11.0)
- DSD TECH SH-U09C2 FTDI adapter to connect to SE-UART

Installing Ozone and J-Link

About Ozone

Ozone is a full-featured graphical debugger for embedded applications. With Ozone it is possible to debug any embedded application on C/C++ source and assembly level. Ozone can load applications built with any tool chain / IDE or debug the target's resident application without any source. Ozone includes all well-known debug controls and information windows and makes use of the best performance of J-Link and J-Trace debug probes. The user interface is designed to be used intuitively and is fully configurable. All windows can be moved, re-sized, and docked to fit the need of any developer.

About J-link

J-Link debug probes are the most popular choice for optimizing the debugging and flash programming experience. Benefit from record-breaking flash loaders, up to 3 MiB/s RAM download speed, and the ability to set an unlimited number of breakpoints in the flash memory of MCUs.

Installing Ozone

Ozone supports both Windows and Linux machines and is available for download at the downloads section of the Segger web pages <https://www.segger.com/downloads/jlink/#Ozone>

Once downloaded, follow the instructions to install Ozone in the Getting Started section of the [Ozone User Guide and Reference Manual](#).

Installing J-link

J-Link supports both Windows and Linux machines and is available for download at the downloads section of the Segger web pages <https://www.segger.com/downloads/jlink/>

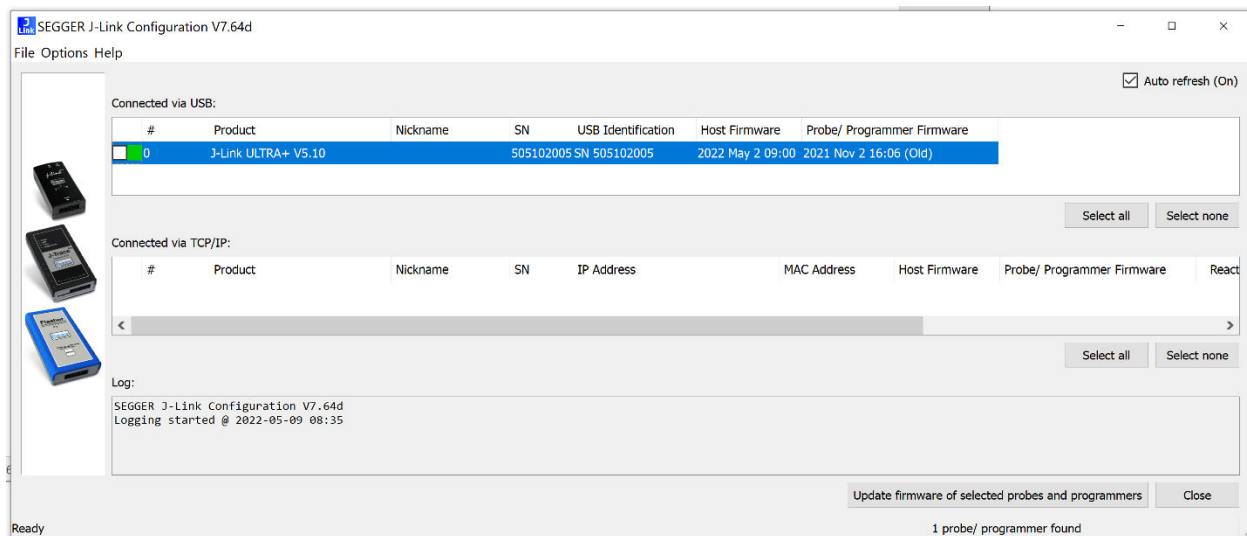
Once downloaded, follow the instructions to install J-link in the Setup section of the [J-Link / J-Trace User Guide](#)

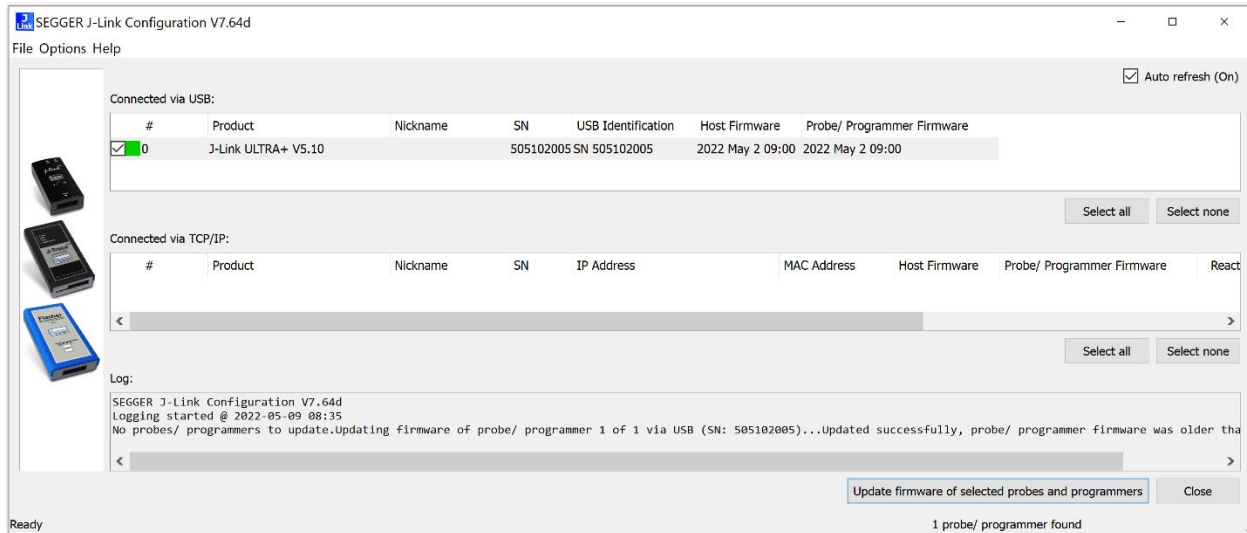
Connecting the J-Link probe

Connect the J-Link probe to the JTAG interface on the Alif CPU board using the 19 PIN connector. The other end of the cable is connected to the J-Link probe. The probe is connected to your machine using the USB cable. Once the probe is connected you should see two green lights lit up on the probe if you have a J-Link Ultra and one green light if you have a J-Link Base.

Configuring the J-Link probe

To ensure that the J-Link probe can be properly identified, configure the probe using the J-Link Configurator. With the J-Link connected, open the J-Link Configurator tool and the tools should recognize and display the connected probe. If the probe firmware is recognized as an old version then select the probe and update the firmware.





Preparing stubs for the M55_HE CPU core

The Blinky examples provided in the Alif CMSIS Ensemble package are designed to run on the M55_HE CPU core. Debug stubs may be needed to be running on the M55_HE CPU core in order for the J-Link to be able to initiate a debug session. If the stubs are not in place the J-Link may not be able to connect to the core to initiate a debugger session.

TIP: if you have previously been running other application SW, the contents of the application MRAM can be erased to avoid any issues. Use command: `Python app-write-mram.py -e APP`

To run the SETOOLS commands, open a command terminal session.

Run the following SETOOL commands:

- `Python app-gen-toc.py -f build/config/app-cpu-stubs.json` (generates the Application TOC)
- `Python app-write-mram.py -r` (writes the Application TOC to MRAM)

To verify the stubs are in place, you can either:

- Using the SETOOLS, run the `maintenance.py` script and choose option 5 – get TOC info which will show the stubs listed as loaded and booted on the relevant CPUs.
- Connect to SE-UART terminal session and reset the device. You should now see the stubs for the M55 cores listed as loaded and booted in the terminal session output.

```
COM5 - Tera Term VT
File Edit Setup Control Window Help

D<'y"BF"0'

SES A1 EVALUATION_BOARD RELEASE v0.45.0 Apr 1 2022 19:07:54
[SES] Device ID = A100
[SES] PLL code version 0.0.4
[SES] LCS-1
[SES] System partition address 0x80580000
[DEV] Vouding Data: 0x00C03FFB
[SES] System partition processed (0x00000000) BL STATUS OK
[SES] Application partition processed (0x00000000) BL STATUS OK

[SES] FC:Rgn - 7:1 7:2 7:3 7:4 7:5 8 Document was last saved: 37m ago 3:1 13:2
[SES] Protected areas:
      0x80580000 - 0x805FFFFF

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Name | CPU | Store Addr | Obj Addr | Dest Addr | Boot Addr | Size | Version | Flags | Time (ms) |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| SERAM0 | CM0+ | | 0x000000E0 | | | 57024 | 1.0.0 | u s | 0.00 | |
| SERAM1 | CM0+ | | 0x000200E0 | | | 57024 | 1.0.0 | u s | 0.00 |
| DEVICE | CM0+ | 0x805C1EE0 | 0x805C14E0 | | | 712 | 0.5.0 | | | 10.69 |
| HE_DBG | M55-HE | | 0x805C21B0 | | | 2256 | 1.0.0 | uLs | 0.00 |
| HP_STUB | M55-HP | 0x8057E400 | 0x8057DA00 | 0x50000000 | 0x50000000 | 2256 | 1.0.0 | uLUB | 9.97 |
| HE_STUB | M55-HE | 0x8057F6D0 | 0x8057ECD0 | 0x60000000 | 0x60000000 | 2256 | 1.0.0 | uLUB | 9.98 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
Legend: <u><C>ompressed, <L>oaded, <U>erified, <s>kippped verification, <B>ooted, <E>ncrypted, <D>eferred

[SES] CM0+ Frequency is 100 MHz
[SES] os Kernel V10.4.2
[SES] Main Task - looping forever...
/
```

Building the Blinky Program file

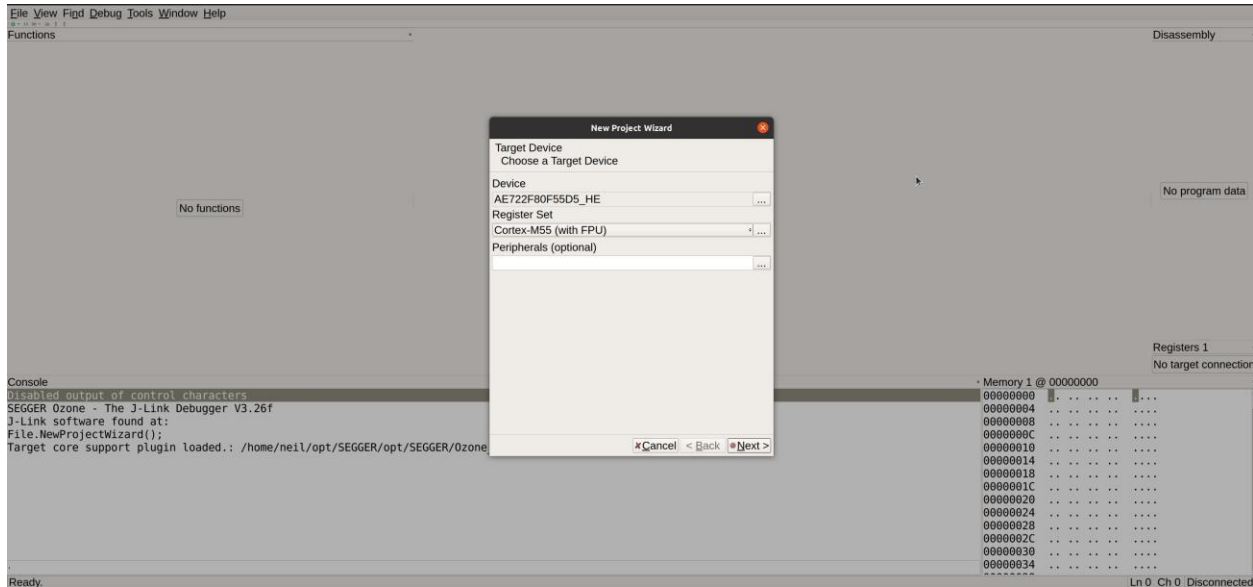
To be able to debug using Ozone you will need an ELF file which contains the executable binaries and the debug symbols. The ELF file can be generated using Arm Development Studio or Keil MDK.

To generate an ELF file using Arm Development Studio, follow the instructions in the “Getting Started with Bare Metal Design using Arm DS” document to compile and build the Blinky project which is part of the Alif Ensemble CMSIS Pack. Once the Blinky project has been built, the ELF file can be found under the debug folder of the project in the Development Studio Workspace folder. The format of the ELF file generated by Arm DS is .axf.

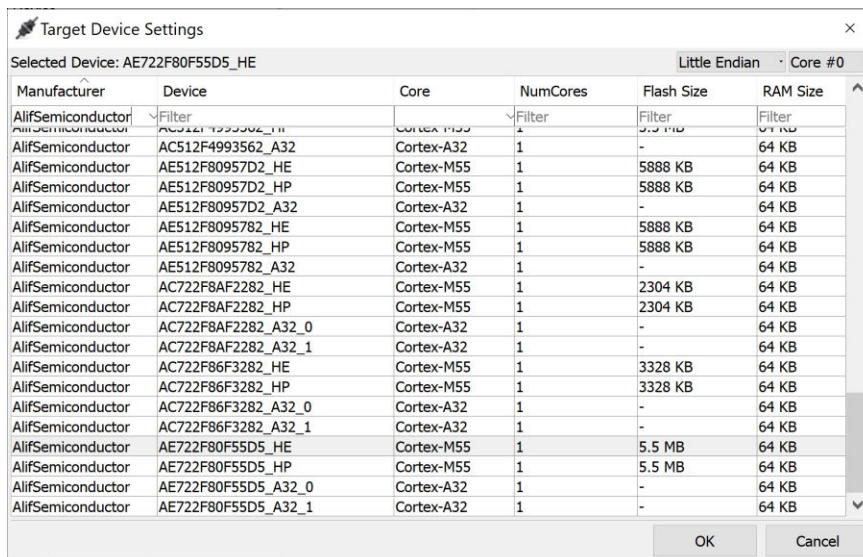
To generate an ELF file using Keil MDK, refer to the Keil MDK web site for instructions on how to set up and build a project using Keil MDK. The ELF file will be created under the objects folder of the project in the MDK workspace. The format of the ELF file is .axf.

Debugging Blinky using Ozone and J-Link

Start the Ozone application and the new project wizard starts automatically. Click on the Device field to open and select the E7 M55-HE CPU as the target device.



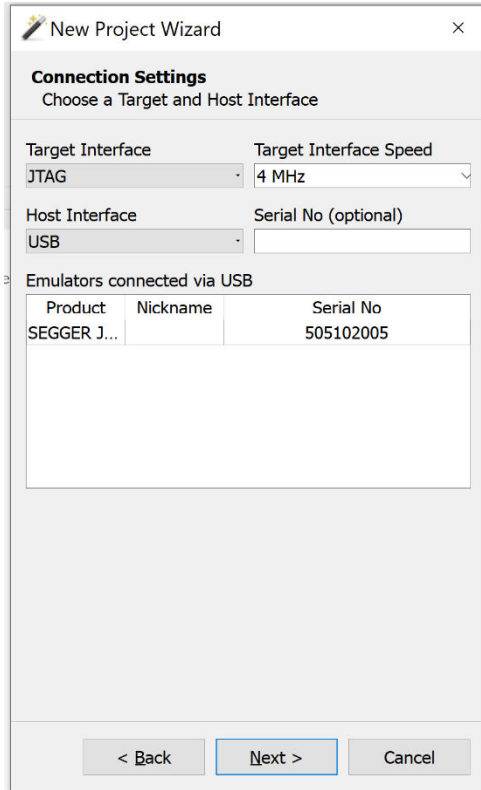
The M55_HE target device can be easily found by selecting AlifSemiconductor in the manufacturer column and selecting AE722F86F55D5_HE in the device column.



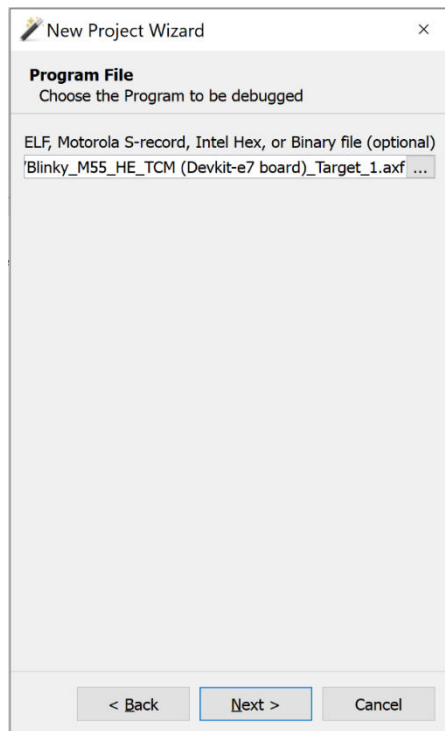
Ozone automatically selects Cortex-M55 (with FPU) register set which is the correct selection for the M55 core.

The Peripherals field can be left blank. Select Next to move to the Connection settings.

With the probe connected it should be identified and the fields automatically filled as follows.



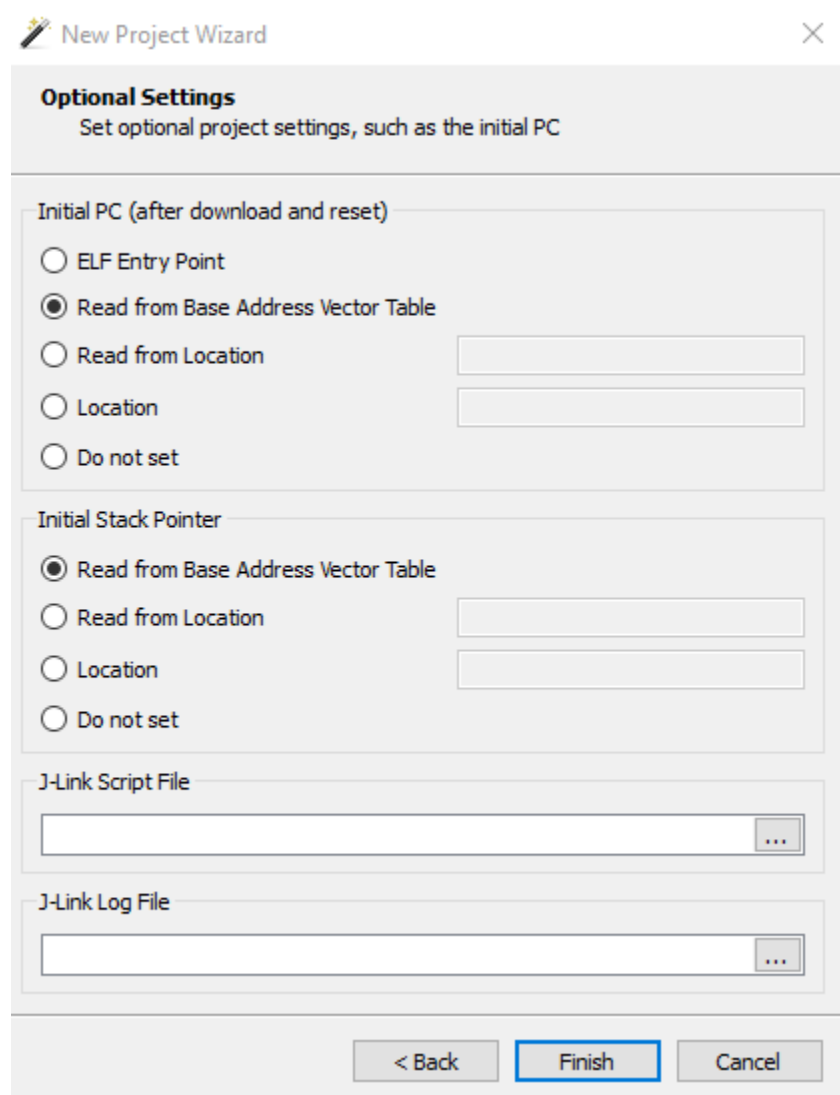
Click next to choose the program file. The axf file for the Blinky project can be found in the Arm DS workspace under the debug folder of the Blinky example project or in the MDK workspace under the objects folder of the Blinky example project.



Selecting Next takes you to the optional settings. For ELF files generated by Arm DS or Keil MDK, change the default settings for the Initial PC to “Read from Base Address Vector Table”.

Note, failing to select the correct settings may result in a hard fault when the debug session is initiated.

Press Finish to complete the set up.

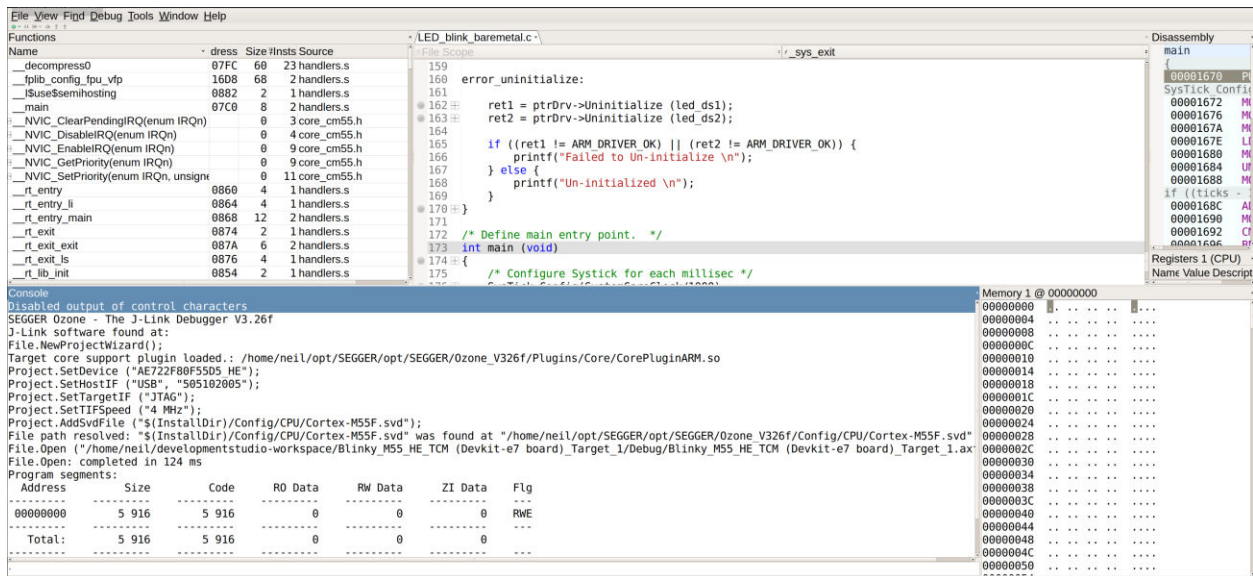


The image shows a screenshot of the 'New Project Wizard' dialog box, specifically the 'Optional Settings' tab. The dialog has a title bar with a pencil icon and the text 'New Project Wizard'. Below the title bar, the tab is labeled 'Optional Settings' with a subtitle 'Set optional project settings, such as the initial PC'. The settings are organized into four sections:

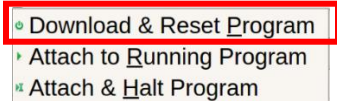
- Initial PC (after download and reset)**: Contains five radio button options: 'ELF Entry Point', 'Read from Base Address Vector Table' (which is selected), 'Read from Location' (with an adjacent text input field), 'Location' (with an adjacent text input field), and 'Do not set'.
- Initial Stack Pointer**: Contains four radio button options: 'Read from Base Address Vector Table' (selected), 'Read from Location' (with an adjacent text input field), 'Location' (with an adjacent text input field), and 'Do not set'.
- J-Link Script File**: A text input field with a browse button (three dots) on the right.
- J-Link Log File**: A text input field with a browse button (three dots) on the right.

At the bottom of the dialog, there are three buttons: '< Back', 'Finish' (which is highlighted with a blue border), and 'Cancel'.

Once the setup is complete the debug session is initiated.

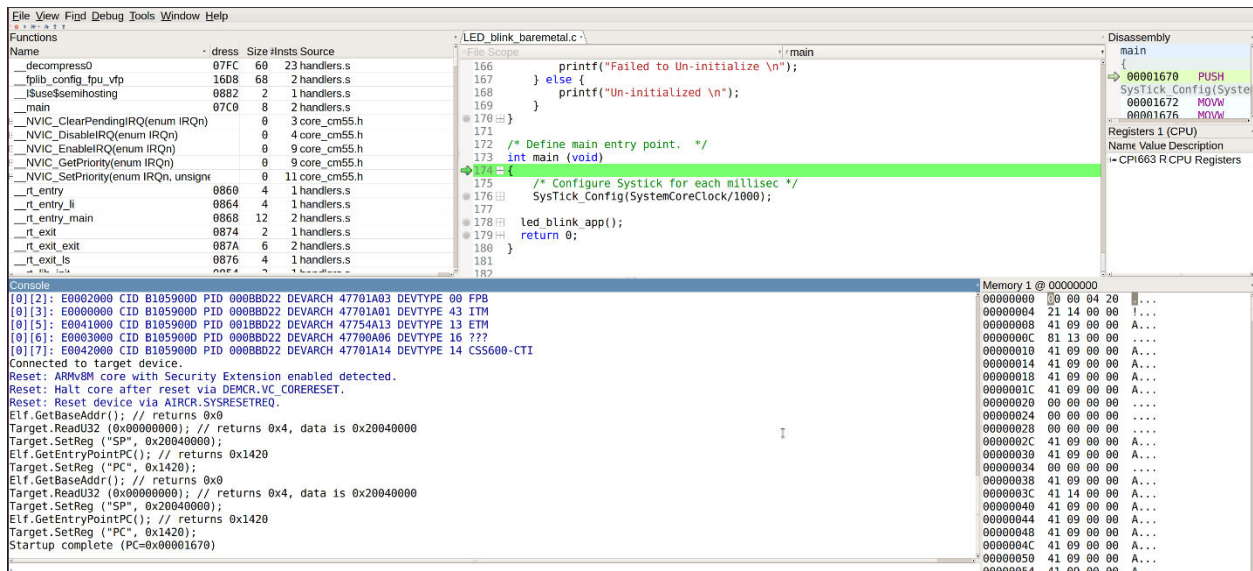


Next, start the debug session from the debug menu. Choose the “Download & Reset Program” option to download the Blinky program and reset.

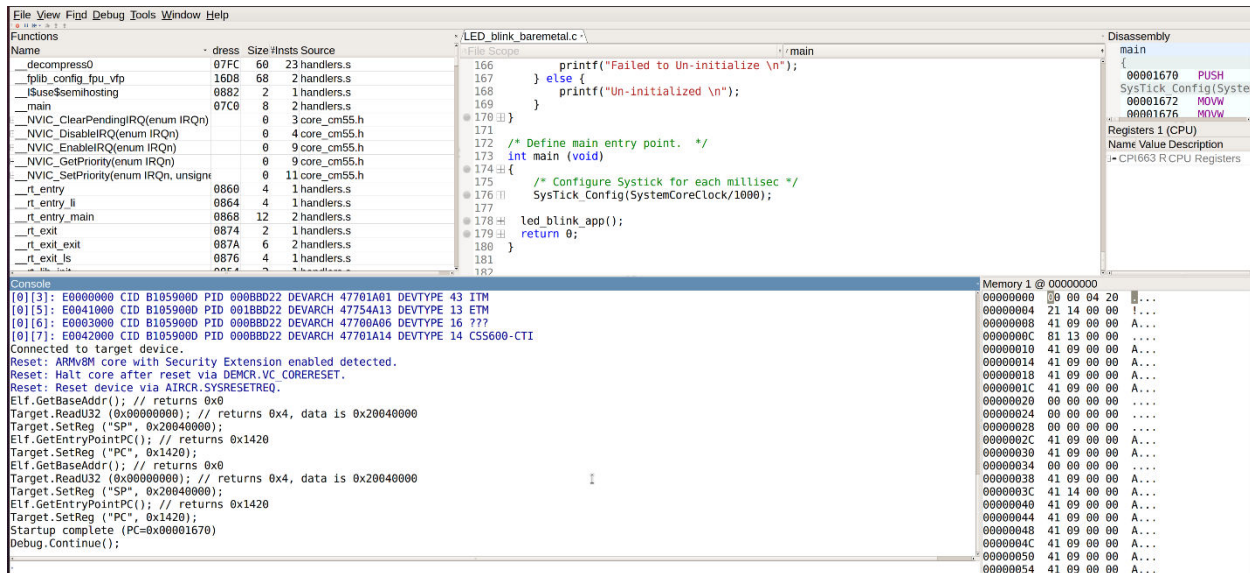


The debug session start-up completes and jumps to main.

From here it is possible to stop, resume the program, set breakpoints, or step through the code.



Pressing resume, the Blinky program executes and you should see the lights blinking on the base board. Controls can be used to halt and exit the debug session or to perform many other debug procedures. Refer to the [Ozone User Guide and Reference Manual](#) for details on the supported debug features and controls.



Adding Segger Ozone and J-Link debug capability to your system gives you a debug platform independent of the design creation environment that can support debug of Cortex-M55 designs targeting the Alif Ensemble Processors created either in Arm DS or Keil MDK.

For further information and tools support downloads for Alif Ensemble processors, visit the Ensemble family web page at www.alifsemi.com/ensemble.

Document History

Version	Change Log
1.0	Initial web release supporting Beta Ensemble Development Kit
1.01	Updated to Ozone 3.26f which adds an SVD-file for Cortex-M55